

Machine Learning

Intelligent Systems II

Mário Antunes

2026

Practice Guide: Machine Learning for Agents

Machine Learning (ML) allows agents to learn patterns from data and improve their performance over time without being explicitly programmed for every scenario. In this guide, you will explore core ML algorithms and their application in agent-based systems.

Part 1: Fundamentals - K-Nearest Neighbors (KNN)

The **K-Nearest Neighbors (KNN)** algorithm is a non-parametric method used for classification and regression. The input consists of the k closest training examples in the feature space.

1.1 Brute Force KNN

Your first task is to implement a simple KNN classifier from scratch using **NumPy**.

Task:

1. Initialize a class that stores the training data (X_{train}, y_{train}).
2. Implement a `predict` method that, for a new query point x_q :
 - Calculates the Euclidean distance to all points in the training set.
 - Identifies the k smallest distances.
 - Returns the majority class among those neighbors.

Hint:

- Use `np.linalg.norm(X_train - x_q, axis=1)` for vectorized distance calculations.
- Use `np.argsort` to find the indices of the k nearest neighbors.

1.2 Efficient KNN with FAISS

In large-scale agent environments, brute force is too slow ($O(N)$). Use the **FAISS** (Facebook AI Similarity Search) library to implement an Approximate Nearest Neighbor (ANN) search.

Task:

1. Initialize a FAISS index using `IndexFlatL2`.
2. Add your training vectors to the index.
3. Use the search method to retrieve the k nearest neighbors.

Observation: Compare the time taken for a query with 10,000 points between your NumPy implementation and FAISS. Which one scales better for real-time agents?

Part 2: The Hot/Cold Number Guessing Agent

Traditional agents use Binary Search to solve this game. In this task, you will create an agent that uses **Linear Regression** to predict the target number based on distance feedback.

Scenario:

The agent guesses a number between 0 and 100 (configurable). The environment returns the **absolute distance** to the hidden target.

Task:

1. Implement an agent that starts with a random guess.
2. After each guess, store the (guess, distance) pair.
3. Train a regression model (e.g., Ridge or LinearRegression) using the stored history.
4. Predict the “zero distance” point to make the next guess.

Hint:

- Your feature X is the guess value, and the target y is the feedback distance.
- To find the best guess, generate a range of candidates (e.g., `np.linspace(0, 100, 1000)`) and select the one that the model predicts will have the minimum distance.

Part 3: Tic-Tac-Toe Move Evaluation

Instead of using a search tree (Minimax), we can train a classifier to recognize **Winning** vs. **Losing** board states.

Task:

Complete the Tic-Tac-Toe agent located in the `04_machine_learning/ex02/` directory.

1. **State Representation:** Map the 3×3 grid to a vector of 9 integers (1 for X, -1 for O, 0 for empty).
2. **Move Evaluation:** For each valid move, simulate the resulting board state.
3. **Prediction:** Pass the simulated state into a trained **Neural Network (MLP)**.
4. **Action:** Select the move that has the highest probability of belonging to the “Win” class.

Hint:

- Use `model.predict_proba(state)` to get confidence scores for [Loss, Draw, Win].
- Review the historical game data provided in the support folder to understand how the model was trained.

Part 4: Hybrid Agent - World Modeling in Mazes

In complex environments, an agent might not see the whole map. We will build a **Hybrid Agent** that uses ML to predict the world layout and **A* Search** to navigate.

Task:

Implement the logic for the HybridMLAgent in the maze simulation.

1. **Feature Extraction:** Look at the 8 neighbors of an unknown cell. Create a feature vector (1 for floor, 0 for wall, -1 for unknown).
2. **Online Learning:** As you explore, update a `RandomForestClassifier` with the newly discovered tiles.
3. **Informed Planning:** When running A*, use the ML model to “guess” if an unknown tile is a wall or a floor.
4. **Cost Adjustment:**
 - If the model predicts “Floor” (Free), assign a standard movement cost.
 - If the model predicts “Wall”, assign a very high movement cost to discourage the planner from going there.

Hint:

- Review the base implementation in `04_machine_learning/ex03/agents/`.
- Focus on the `_get_features` and `_get_movement_cost` methods.

Goal: Can your hybrid agent find the exit of an unknown maze faster than an agent performing random exploration? Compare their “steps taken” metrics in the simulation frontend.